Vibe Coding 真解

目录

序言 **iii**

第一章　问题不是“能生成”，而是“能负责” **1**

第二章　效率之外 **2**

第三章　优雅的代码 **4**

第四章　始于乐趣，成于责任 **6**

第五章　复杂性预算 **8**

第六章　提问即设计 **10**

6.1　接口实现任务 11

6.2　重构任务 11

6.3　需求澄清任务 12

第七章　评审优先 **13**

第八章　协作公约 **15**

8.1　文本化与协议驱动 15

8.2　模块化隔离与边界控制 16

8.3　精准调试与最小复现 16

8.4 评审权威与架构守门 16

第九章 收放之道 **18**

9.1 什么时候可以尽情 Vibe 18

9.2 什么时候必须切回工程纪律 19

9.3 一套可执行的切换规则 19

9.4 收束结论 19

结语 **21**

序言

这本书来自我对生成式人工智能辅助编程的长期观察。过去几年，大语言模型已经深度进入代码生成、问题求解、系统搭建和交互设计。编程活动也随之发生了明显变化。变化首先出现在工具层，同时也进入了开发者的工作方法里：怎么提问，怎么判断，怎么组织系统，怎么为结果负责。

所以，这本书不会写成一部传统意义上的编程教程。它不以语法、关键字和基础语句为中心，也不沿着“从入门到精通”的路线展开。原因很直接。语法知识已经有很多成熟资料，今天开发现场里更难的问题也不在这里。开发者真正经常面对的，是另外几件事：怎么把一个模糊想法推进成可运行原型，怎么把一个原型收敛成可维护系统，怎么在工具能力不断增强的情况下，依然保留稳定的判断。

基于这个前提，本书把“代码”看成一种兼具执行功能和认知功能的中介。代码当然要运行，但它也会留下问题建模、边界划分、抽象组织和决策取舍的痕迹。代码既是结果，也是思考过程的沉淀。编程也就不只是规则记忆和语法拼装。它更像一个持续推进的过程：构造、验证、修正、再组织。开发者不断把自己的认知外化出来，再交给现实检验。

Vibe Coding 也要放在这个背景下理解。这里说的 Vibe Coding，不只是借助人工智能更快写代码，也不只是“用自然语言换代码”的低门槛操作。生成式工具广泛介入之后，变化最大的地方，是软件实践里的判断、审阅、约束和责任分配。Vibe Coding 更接近一种新的组织方式：高频生成、持续审阅、结构约束和责任承担同时发生。

这也决定了本书的写法。我不打算按传统教材的方式，从语法基础一路讲到项目应用。我更关心问题建模、系统骨架、协作语义、迭代节奏和结构治理。因为在人工智能已经能够大规模生成实现细节的情况下，真正稀缺的能力已经变了。开发者现在更需要整体结构的把握能力、复杂度走势的感知能力，以及对结果是否可维护、可解释、可演化的判断能力。缺了这些能力，项目就算很快得到一堆能跑的代码，也很容易在短时间内失控。

本书主要围绕四个问题展开。第一，人工智能已经能快速生成实现细节，开发者为什么还要维护系统的概念完整性。第二，效率被放大之后，项目为什么更容易积累复杂度。第三，什么样的代码可以叫作适合 Vibe Coding 场景的“优雅代码”，这套标准怎么真正落地。第四，一个出于兴趣启动的项目，为什么一旦进入公共使用情境，就会变成一个需要承担责任的工程对象。这几条线会

贯穿全书。

最后说明一点。本书不打算给出终局答案。它更像一组从实践里提炼出来的方法判断。这些判断是否有效，还要靠读者在具体现场中检验、修正和补充。书里有明确的立场，也有明确的问题意识。只要它能促使读者重新思考几件事：什么叫真正的开发，什么样的代码值得保留，在高速生成的条件下哪些判断责任仍然不能让渡，那么这本书就写到了点子上。

第一章　问题不是“能生成”，而是“能负责”

过去一年里，AI 编程最明显的变化，是写代码的起点被大幅提前了。一个想法从口头描述到原型落地，常常只需要几轮对话。很多过去要花半天、一天，甚至几天才能完成的搭建工作，现在可以在更短的时间里完成。

但工程现场真正关心的问题，不是“能不能很快做出来”，而是“做出来的东西能不能长期成立”。代码能跑，不等于系统可靠。AI 很擅长给出局部可用的实现。它能快速拼出函数、页面、接口和测试样板，也能给出一套看起来相当完整的初稿。问题在于，系统最难的部分通常不在局部，而在整体一致性。Brooks 讲“概念完整性”，到今天仍然准确。一个系统能不能长期演化，关键不在某个函数写得有多聪明，关键在目标、术语、边界和约束能不能在多轮迭代里保持一致。

一句话生成贪吃蛇，就是一个典型例子。十分钟内做出可玩的版本并不难。但再往下测，问题很快就会冒出来。难度曲线可能异常，速度增长和渲染帧率绑在一起，机器稍微忙一点就忽快忽慢；输入状态可能穿透，连续按键时触发非法反向。这些问题偶发，很难复现。又或者界面事件污染核心逻辑，窗口缩放直接触发全局重置，整个交互被打断。这些问题都不是简单的语法错误。问题出在系统要求没有定义清楚。模型能补代码片段，但它不会主动替我们把行为边界定好。

所以，AI 时代开发者的角色变了。过去，开发者主要负责亲手实现。现在，开发者越来越需要负责三件事：定义规则、审阅结果、维护边界。一个更稳妥的工作流通常分三步。先把系统规则写清楚，包括输入输出、状态边界和失败语义。再让模型生成候选实现。最后用测试和评审决定哪些内容可以进入主线。这里最关键的事，不是让模型多写一点，而是让系统在每一轮生成之后，仍然保持可判断、可解释、可拒绝。

这种操作只是将 AI 的能力作用于合适的位置：高效生成候选稿。至于什么是正确实现，什么值得长期保留，什么只是临时拼凑，什么会在后续演化里放大复杂度，这些判断仍然要由开发者来做。Vibe Coding 走向成熟之后，生成会越来越容易。真正拉开差距的，是我们到底能不能守住判断边界和责任边界。

第二章　效率之外

上一章讨论的是概念完整性。这一章重点回答另一个问题：效率被放大之后，项目为什么反而更容易在更短时间里失控。今天最需要重新理解的一点，就是“更快”本身不自动等于“更好”。很多时候，更快只是在更短时间里暴露问题，也在更短时间里固化坏结构。

从软件工程的经验看，项目生产力从来不是一条可以无限上升的直线。一个系统在初期推进很快，模块少，边界清，耦合浅，历史负担也轻，因此新增功能经常像是在空白地带上展开。但随着项目持续演化，依赖关系开始增加，命名和抽象的语义慢慢漂移，早期为了方便接受的妥协也会沉积成结构惯性。到了某个阶段，团队会普遍感到难受：投入没有减少，人手甚至在增加，整体推进速度却开始下滑。一个看似很小的改动，会牵出多处不确定影响；一个原本简单的修复，也会因为既有耦合太重而变得昂贵。系统的复杂性让处理更加棘手。

生成式人工智能改变了这种退化发生的速度，也改变了它的显现方式。模型确实能帮助开发者快速搭建页面、生成接口、补全测试、整理样板代码，短时间内显著提高代码产出量。但代码数量增加，不会自动带来结构质量提升。相反，一次草率的依赖引入，一次未经充分论证的抽象提升，一段职责混杂但暂时能跑通的实现，在生成式协作环境里都会更容易被沿用、复制和扩展。原本只是局部瑕疵的设计选择，经过几轮生成之后，很快就会变成默认模式，并进一步沉积成整个代码库的结构倾向。

所以，人工智能在开发流程里的作用，应该看成一种放大机制。它可以放大正确路径，也可以放大错误路径，而且两边的速度一样快。这样一来，“效率”这个词就必须重新定义。只看单位时间里生成了多少代码，生成式人工智能当然提高了表面效率；但工程上的效率从来不只和当前产量有关，它还和未来维护成本、结构稳定性、演化自由度直接相关。一个系统就算能在短期内迅速交付新功能，只要这种交付方式在持续透支可维护性，这种效率就只是局部时间尺度上的幻象。

今天省下来的一次重构，往往会在明天以更高代价返还。此刻为了赶进度而接受的含混实现，也很可能在不久之后变成最难拆除的关键障碍。技术债真正危险的地方，在于它总以“暂时方便”的形式出现，然后在系统成长之后用更高利息回收成本。开发现场里最具破坏性的说法之一，就是“先这样吧，后面再改”。多数项目里，“后面再改”都不会自动发生。一种不良结构只要以默认形式进入主干，它就会在后续迭代中不断吸附新的依赖，最后变成很难逆转的既定事实。

这一点也可以从软件文化里的**破窗效应**来理解。系统的第一处破损，往往并不刺眼。它可能只是一次命名不一致，一个略显多余的外部依赖，一段职责边界不够清晰的函数，或者一次未经说明的例外处理。但在生成式协作环境里，模型很容易沿着既有模式继续扩写。团队只要对第一个例外保持沉默，后续输出就会迅速把例外常态化，最后让它变成系统风格的一部分。KISS 原则在人工智能时代依然重要，而且更有现实针对性。保持简单，就是在维持结构的可解释性。

从认知机制看，这个问题还和“**快思考—慢思考**”的区分密切相关。模型输出天然迎合快速反应系统：答案很快，叙述流畅，结果也经常具备足够强的即时可用性。开发者在这种低阻力状态下，很容易直接接受结果，把看起来合理当成已经成立。要避免这个问题，就必须主动建立触发慢思考的机制：要求关键结构改动说明必要性，要求新依赖证明引入收益，要求抽象提升解释边界所在，要求每次较大规模生成之后都附带最小可回归验证路径。真正的克制，就在这里：约束，约束。

所以，这一章谈“效率之外”，重点不是反对效率，而是修正对效率的理解。真正值得追求的效率，是在单位时间里增加更多确定性，减少更多未来返工，同时给系统保留更多可演化空间。开发者只要还能持续对模型输出说“不”，并且把这份否决权稳定地用在维护概念完整性、控制复杂度累积和压缩不可解释区域上，人工智能就能真正从产出机器变成能力放大器。

第三章　优雅的代码

前两章讨论了两个前提性问题。第一，人工智能高速生成的条件下，开发者仍然要维护系统的概念完整性。第二，效率被放大的环境里，开发者要具备对低质量便利持续说“不”的能力。什么是“优雅的代码”，以及如何使用还需更深入的探讨。

这里先要把“优雅”说清楚。优雅不是一个单纯的审美词。它不只是排版整齐，不只是语句漂亮，也不只是符合某种个人风格。软件工程里，优雅首先是一种生存策略。它的目标很明确：控制维护成本，降低认知负担，延缓系统熵增，让局部变化尽可能在局部闭合，避免轻易引发全局扰动。优雅的代码让系统在时间维度上仍然可理解、可修改、可替换。

判断一段代码是否优雅，最直接的办法，就是看它在变化到来时是否还能保持结构稳定。某个需求变更出现后，开发者如果能够较快定位相关模块，说清职责边界，把影响限制在有限范围内，系统通常就还健康。相反，如果只是修补一个很小的逻辑，却必须同时翻阅多个文件、触及多个层级、担心未知副作用，还对既有耦合充满顾虑，那么系统暴露出来的就是整体脆弱性。优雅最终要回到一句话：后续变化是否可控。

这一点可以借助 Unix 传统来看。Unix 工具能长期保持生命力，原因很清楚：单一职责、清晰接口、可组合、可替换。把这套思路放到 Vibe Coding 上，一个重要判断就很自然地出来了：开发者不要试图在一次提示里要求模型同时完成业务逻辑、交互设计、架构组织、异常处理和扩展策略。这样的“全能提示词”通常只会换来一个表面完整、内部混杂的结果。职责大量堆叠在单一对象里，状态流、渲染层、输入处理和策略逻辑彼此穿透。初看省事，后续维护会越来越重。传统工程一直警惕“上帝对象”，原因就在这里。

所以，优雅代码的第一项要求，是原子化。这里说的原子化，不是为了形式上的碎片化，而是要求开发者把不同性质的问题分配到足够小、职责单一的单元中。这样，每个单元都能在有限上下文里被理解、被测试，并在需要时被局部替换。以交互型应用为例，状态推进尽量留在纯逻辑层，渲染层只负责把状态转换成可见输出，输入处理层只负责收集事件并翻译成统一信号，持久化和外部通信则划到边界模块里。这样做的价值非常实际：规则变更不会直接改动渲染机制，界面调整不会反向污染核心状态推进，输入方式变化也不用重写业务逻辑。

和原子化配套的，是接口先于实现。很多生成式协作中的混乱，都出在开发者还没有明确输入、

输出、错误语义和边界条件，就急着向模型索要“完整方案”。目标模糊时，模型给出的结果大多只是统计意义上的合理拼接，工程上却未必稳定。更可靠的做法，是先把模块契约界定清楚：接收什么、返回什么、什么情形下失败、失败时暴露什么信息、哪些状态可以改写、哪些约束必须始终保持。接口的意义就在这里：压缩歧义，收敛生成方向，降低后续返工概率。

优雅代码还要求开发者重新认识“文本”在协作链条中的基础设施地位。无论是 Markdown 形式的需求草稿，JSON 形式的结构化配置，还是对模块契约、变更原因和边界条件的说明文字，文本都有很强的工程价值：可读、可审、可版本化、可比较、可回滚。完全依赖一次性对话上下文来驱动模型生成，协作过程会很脆弱。文本化约束更有助于形成可复现、可追责的人机协作流程。开发者可以用文本定义目标和边界，用模型生成候选结果，用脚本执行规范化变更，再用日志记录每次修改背后的决策依据。这样，项目会慢慢从即时语境里拼出来的产物，变成一个过程透明的工程对象。

再往下走，优雅实现还离不开“机制与策略分离”。机制回答的是“系统如何做事”，策略回答的是“系统在什么条件下这样做”。两者一旦混在一起，系统很快就会失去弹性。生成式协作里的常见问题之一，就是模型在实现机制时顺手把大量业务策略也写死进去，导致规则一旦调整，底层实现就得整体改动。更稳妥的方式，是让机制层尽量保持中立和可复用，把权限分级、审批条件、例外处理、频率限制、优先级排序这些内容放到策略层或配置层里控制。机制越纯净，策略越容易演化，系统的可维护性和可扩展性也就越强。

综合来看，“优雅代码”可以概括成几条彼此勾连的结构约束：职责尽量单一而且说得清楚，接口在实现之前先被界定，文本承担必要的协作和追责功能，机制和策略清楚拆开，系统里的复杂度在每一轮迭代中被主动回收。这里还要强调一点：优雅不要求初稿完美。多数实践里，初稿都会粗糙、不平衡，甚至混乱。真正关键的，是开发者能不能始终把初稿看成候选稿，而不是定稿，并且在后续阶段持续进行审阅、删减、拆分和重写。只要这种能力仍然存在，优雅就不是一句抽象修辞，它会变成系统得以长期存活的现实条件。

第四章　始于乐趣，成于责任

很多项目，最初都来自一种很朴素的冲动——“这个想法值得试一试”。以兴趣为起点的项目形态非常普遍。在很多场景里，它也是创新实践最真实的源头。

兴趣驱动在软件实践里有很高的价值。和完全由外部期限、任务指标推动的开发活动相比，出于个人好奇心和问题意识展开的项目，往往更容易带来高密度提问、更频繁试错，以及更强的内在持续性。很多后来被视为重要系统的工程实践，起点都只是某个具体问题上的反复折腾和持续尝试。“好玩”这个动机值得认真看待。对 Vibe Coding 来说，这一点尤其重要。生成式人工智能提供了一个成本很低的实验环境，开发者可以围绕直觉迅速搭建原型，并在多轮交互里不断逼近问题边界。

但兴趣驱动有自己的天然边界。兴趣可以提供启动动力，却不足以自动构成工程终点。一个项目只要被他人复用、被团队接管、被流程依赖，甚至只是被未来的自己反复调用，它就已经不再是纯粹私人性的试验品，而开始获得公共属性。进入公共使用情境之后，软件就要面对稳定性、可解释性、兼容性和迁移成本等问题。一个项目还能不能被叫作“玩具”，关键不在功能数量，也不在代码规模，关键在于有没有形成他人对它接口、行为、输出格式和可持续性的依赖。依赖关系一旦成立，责任关系就会随之成立。

人工智能时代放大了这个转折。生成式工具显著压缩了从想法到原型的时间间隔，开发者更容易把生成完成当成开发完成，把演示成功当成可以交付。这种误判在个人尝试阶段也许只是带来局部返工，但在协作、开源或产品环境里，影响会迅速扩大，最后变成信任损耗。今天为了追求速度而接受的含混命名、重复逻辑、隐式依赖和未经验证的边界分支，明天就会变成别人维护时最昂贵的负担；此刻看似无伤大雅的一次接口调整，也可能打断多个下游使用者的正常流程。兴趣不会自动豁免责任，生成速度也不会自动抵消后果成本。项目从私人世界进入公共世界之后，开发者就必须重新理解自己和作品之间的关系。

“负责任的 Vibe”更像一种实践原则。它不要求开发者在项目初期就用大型商业系统的标准做全量治理，也不要求所有探索性实践提前背上过重的文档和流程负担。它真正要求的，是开发者能识别项目所处阶段，并据此调整治理强度。实验阶段可以容许高频试错、快速原型和一定程度的局部粗糙。主干阶段就要引入明确的结构约束、节奏控制和质量底线。草稿可以粗糙，但草稿不能伪装成成品。实验分支可以追求速度，主线分支必须追求稳定。

从工程实践看，这种责任转折至少要落实为几条最低治理要求。关键路径需要保留最小可读测试，确保核心行为不会在连续修改中无声退化；接口变更要附带影响范围说明和迁移路径提示，避免下游依赖在无预警的情况下被打断；命名和抽象要优先服务长期可读性，而不是当下生成时的局部可用；暂时解释不清但先合并的代码，也要设置明确的重审机制，不能让它们以默认状态永久留存；重要依赖的引入原因、替代方案和潜在退出成本，也应该在项目达到一定规模后被记录下来。这些要求看起来不复杂，但我们未见能坚持下来。

这一转折也改变了开发者在人工智能协作里的角色结构。随着生成能力普及，代码产量已经不再是最稀缺的资源。今天更稀缺的，是标准制定能力、边界判断能力和风险识别能力。开发者越来越像结构师、审阅者和责任承担者：模型负责批量生成，开发者负责筛选和收敛；模型负责沿上下文补全实现，开发者负责识别哪些内容真正符合系统主线；模型负责提供统计意义上的流畅答案，开发者负责把这些答案重新压缩成工程意义上的可解释结构。

所以，乐趣和责任不是对立的两极，它们更像同一条实践路径上的前后两个阶段。乐趣提供创造冲动和探索韧性，让项目得以启动；责任提供工程护栏和公共承诺，让项目能够进入他人可依赖的世界并长期存续。缺了前者，开发很容易退化成机械劳动；缺了后者，开发就会停留在情绪化的即时生成层面，很难形成可持续作品。Vibe Coding 走向成熟之后，关键任务就是用责任机制承接兴趣，把兴趣真正转化成长期有效的工程能力。

第五章　复杂性预算

生成式人工智能深度介入开发流程之后，代码生成的边际成本明显下降了。样板结构、接口草案和局部实现，都能在很短时间里得到，项目启动速度也明显提升。但工程压力没有同步消失。代码一旦进入系统，理解、修改、测试、协作和维护仍然需要长期投入。生成提速改变了产出节奏，维护成本的基本结构没有变。

所以，有必要引入“复杂性预算”这个工作概念：一个项目在特定阶段内，究竟能承受多少结构复杂度。复杂性本身无法被彻底消除，系统始终要处理状态、边界、异常、依赖和演化。开发者真正能够管理的，是复杂性进入系统的速度、位置和规模。这个概念在 AI 时代尤其关键，因为模型可以不断给出“看起来可用”的实现。团队只要缺少筛选机制，技术债就会以更高速度累积。复杂性预算的意义就在这里：它帮助团队控制复杂性进入主干的方式。

再往前一步看，复杂性预算本质上就是结构容量管理。每个阶段都只有有限的理解带宽、测试带宽和重构带宽。带宽一旦耗尽，团队就算继续提交功能，也会明显感到交付变慢、回归增多、修改不确定性上升。很多项目到了中后期，表面上还在迭代，实际上已经越来越难改。问题不在于工作不够努力，而在于预算长期超支，却从来没有被显式识别和管理。

为了让这个概念真正落地，这里给出四项最低保留标准，作为 AI 代码进入系统前的基本门槛。

1. **可读性**。维护者能否在短时间里说清这段代码的意图、输入输出和关键路径。答案不明确时，即便代码能运行，也应该先重写再合并。
2. **职责边界**。每个函数和模块都要对应一个主要任务。跨层级堆叠、单点承载多类变化来源的实现，需要优先拆分。
3. **重复控制**。对已有知识的再次复制，要视为预算泄漏。凡是同一规则分散存放在多处，或者为了局部便利引入重复实现的情况，都应该优先收敛。
4. **架构服从**。新代码进入系统时，要在既有结构中有明确位置，并且遵循既有边界与接口习惯。局部写得顺，不能成为偏离主线的理由。

这四项标准不追求完美主义。它们只是帮助团队回答一个基本问题：这段代码值不值得被系统

长期保留。举个例子。参数校验逻辑如果把类型判断、范围判断、默认值回填和错误信息全部揉在几十行嵌套条件里，虽然能跑，但读者很难迅速定位失败条件。更稳妥的写法，通常是拆成基础格式校验、业务规则校验和错误映射三个小函数。再比如一个提交接口里，如果把参数解析、权限判断、业务计算、数据库写入和消息通知全部塞进同一个处理函数，那么任何一点改动都得回归整条链路。短期看着方便，长期维护成本几乎一定会越来越高。

只有标准还不够，还要识别“超预算信号”。实践里，至少有三类信号值得立刻触发重审。第一，同一需求的改动需要跨越过多文件，而且很难定位主修改点。第二，新增功能必须先修补多处历史例外才能落地。第三，一次改动反复引入非预期回归。只要出现这些信号，通常就说明系统局部复杂性已经超过可控阈值，继续叠加新实现只会放大风险。

在流程上，复杂性预算要落实成一个固定闭环：把 AI 输出视为候选稿，按上述标准审阅，对超预算部分直接拒绝，并在合并前完成必要重构。为了让这个闭环真正运转，可以配一份很轻量的检查清单：命名能不能直接表达意图；有没有可合并的重复逻辑；边界条件是不是处理清楚了；新增依赖是不是确有必要；该改动在现有架构中的位置是不是明确。检查项不需要复杂，但必须稳定执行。多数预算管理失败，不是因为没有原则，而是因为检查动作被直接跳过了。

复杂性预算还牵涉一个容易被忽略的问题：预算分配。并不是所有模块都该承担同等复杂度。靠近核心业务和高频变更路径的模块，要保持更低的复杂度阈值；外围适配层可以容纳一定技术性复杂度，前提是边界清楚而且可替换。复杂度一旦压到核心区，整个系统的演化成本都会上升；复杂度更多留在边缘区，整体稳定性会更高。

实践里最常见的错误，就是把“以后再清理”当成默认策略。这个策略在多数项目里都会失效。系统持续迭代时，旧问题会和新需求叠加，修复窗口迅速收窄。AI 提升了产出速度，也放大了拖延成本。所以，复杂性预算不该被看成额外负担，它应该被看成 AI 编程阶段最基础的约束。只有在持续审阅、及时拒绝和主动重构的前提下，生成能力才会真正转化成长期生产力。

第六章　提问即设计

当生成式人工智能进入开发主流程之后，Prompt 已经不再只是“输入一句话试试看”的辅助动作。它正在变成工程产出的上游环节。我们知道复杂性控制从提问阶段就已经开始了。提问方式决定模型的搜索空间，搜索空间决定候选实现的结构质量，结构质量再决定后续维护成本。Prompt 因此应该被纳入工程治理。

“提问即设计”的意思很清楚：Prompt 需要被当作可审阅、可迭代、可复用的设计文档来管理。团队如果把 Prompt 当成一次性对话，产出很容易依赖个人即时状态，结果波动大，可追溯性弱，复用成本也高。团队如果把 Prompt 当作文档，目标、约束、边界和验收条件就会在生成之前被明确下来，后续评审和重构也就有了稳定依据。这个转变看上去只是写法变化，实际上会直接影响系统演化的可控程度。

这一判断和“快思考—慢思考”的认知框架高度一致。模型天然擅长高速度模式补全，开发者在赶进度时也容易依赖直觉输入。两边同时走“快路径”，输出往往很流畅，结构里却容易留下空洞。Prompt 一旦文档化，生成前就等于显式启动了审查机制：先定义问题，再限定边界，再规定验收，最后进入实现。这样做会降低即时便利，但能换来更高的长期稳定性。

从工程角度看，Prompt 文档化至少带来三项直接收益。第一，概念一致。团队成员围绕同一套术语和同一组边界协作，减少每轮生成都在换世界观的漂移。第二，决策可追溯。后续出现回归或偏差时，可以回看当时的约束和目标，定位问题来源。第三，资产可复用。高质量 Prompt 可以沉淀成模板，在相似任务里反复使用，减少重复试错。

为了让“提问即设计”真正落到执行层，这里给出一套六段式 Prompt 结构。它不是唯一答案，但在多数工程场景里足够稳定，也不错。

1. **Context（上下文定义）**：明确系统环境、任务类型、角色视角和当前阶段。
2. **Goal（目标声明）**：把“想做什么”写成可判断的输出目标。
3. **Constraints（边界与约束）**：显式列出禁止项和硬约束。
4. **Interface & Data（接口与数据约定）**：提供输入输出格式、关键数据结构、异常语义和边界样例。

5. **Process（执行路径）**：要求模型按固定顺序产出结果，不要一次性倾倒全部实现。
6. **Review（验收与回归）**：明确检查项和拒绝条件，把评审标准前移。

下面给出几个简化示例。

6.1　接口实现任务

闲聊式提问经常会写成：“帮我写一个订单提交接口。”

更接近设计文档的写法可以是：

> 项目为分层架构，当前需要新增订单提交能力。目标：在 service 层实现订单创建逻辑，并在 controller 层提供统一响应。约束：不新增三方依赖；保持现有 DTO 命名；写入失败需返回可追踪错误码；禁止在 controller 写业务规则。输入：OrderCreateRequest；输出：OrderView。过程：先给模块划分，再给核心代码，再给测试边界。验收：可读性清楚、职责单一、无重复规则、接口兼容现有版本。

两种写法都可能得到“可运行代码”。第二种写法更容易得到“可合并代码”。区别就在于输入是否更像工程问题。

6.2　重构任务

很多团队会直接输入：“这个函数太乱了，帮我优化一下。”

更有效的方式，是先把问题说清楚：

> 当前函数承担参数校验、业务计算、数据库写入和日志记录四类职责。目标：拆分为可单测的三个函数，保持外部接口不变。约束：不改变返回结构；日志语义保持一致；复杂度不高于当前版本。过程：先给拆分方案与调用关系，再给重构实现，再列出回归测试点。验收：修改边界清晰，失败路径可追踪，重复逻辑被收敛。

这样的 Prompt，在生成前就已经完成了相当一部分评审工作。它不是让模型自由发挥，而是在明确边界后邀请模型协作。

6.3　需求澄清任务

当需求本身含糊时，最好的 Prompt 往往不是直接要求代码，而是先要求澄清：

> 你先不要写代码。请基于以下业务描述提出 8 个澄清问题，覆盖目标用户、核心流程、失败场景、性能约束、兼容范围和上线节奏。待我回答后，再输出实现方案。

这个步骤看上去变慢，实际上经常能减少后续大幅返工。提问阶段多投入十分钟，常常可以省掉数小时无效实现。

在团队协作里，Prompt 文档化还需要版本治理。比较实用的做法，是遵循三条轻量规则：Prompt 和代码同生命周期管理；Prompt 迭代要有变更理由；模板应根据任务类型分层，不要一套表格套所有场景。同时也要警惕过度模板化带来的僵化。结构化的目标，是提升判断质量，不是制造文书负担。真正有效的方式，始终是骨架稳定，细节有弹性。

概念治理决定系统主线，效率约束避免短期冲动，结构原则控制实现质量，复杂性预算约束进入速度，而提问即设计，把这些约束前移到了生成源头。高质量 Prompt 本身就是设计活动的一部分。它既是生成指令，也是架构边界的表达；既服务当前产出，也服务后续协作。

第七章　评审优先

在 AI 编程时代，代码生成的边际成本持续下降，理解和维护成本却依然高昂。开发活动的主能力因此发生了明显转移：手工敲代码的稀缺性下降了，结构判断和质量把关的稀缺性上升了。Code Review 也随之发生变化。它不再只是提交流程里的一个环节，而是在逐渐变成编程活动的第一基本功。

这个变化并不抽象。模型可以在几分钟里产出数百行实现，但团队要对这些代码承担长期责任。代码一旦合入主干，后续每一次功能迭代、缺陷定位、性能排查和架构调整都会和它发生关系。AI 可以负责高速产出，人类开发者必须保留对每一行代码的解释权和否决权。边界越清楚，系统越稳定。

《人月神话》的核心结论在今天仍然很适用。Brooks 早就提醒过，协作复杂度会随着参与者数量增加而迅速上升。AI 在工程实践里，可以看成一个高产的“实现参与者”：它能持续提交候选代码，却不会对系统整体一致性负责。团队只要缺少严格评审，局部冲突就会快速积累，沟通成本和返工成本也会同步上升。评审机制在这里承担的角色，就是控制复杂性输入速率的人机闸门。

基于这个前提，AI 时代的 Code Review 至少要覆盖以下四个维度。

1. **正确性（Correctness）**：代码是否真正解决了需求，并覆盖了关键失败路径。
2. **可读性与透明度（Readability）**：维护者能否迅速说明代码意图、依赖关系和控制路径。
3. **可维护性（Maintainability）**：提交是否引入了重复逻辑、跨层职责混写或新的技术债。
4. **可演进性（Evolvability）**：这段实现今天能运行，明天的变化是否还有合理空间。

在正确性维度上，最常见的问题是主路径可运行，但边界条件缺失。评审时要检查是否存在未声明副作用，异常语义是否完整，并尽量用最小可复现测试去验证关键路径。不能只满足于“看起来差不多”。在可读性维度上，评审要优先看意图是否清楚：命名是否一致，是否存在魔法数字，控制流是否被过度压缩，关键分支是否对维护者可见。模型有时会用技巧性写法压缩代码长度，这种“简洁”经常只是把理解成本转嫁给未来。

在可维护性维度上，评审要格外警惕 AI 通过复制局部逻辑来快速满足需求的倾向。这类“短平快”实现最容易触发破窗效应。金额格式化、状态映射、权限判断这些规则，只要在不同文件里出现多个相似但不完全一致的版本，后续修改就会变得很脆弱。评审时要优先要求抽取公共逻辑，并补齐最小边界测试，让规则只有一个事实来源。

在可演进性维度上，评审需要保护未来变化能力。AI 可能为了满足当前任务而引入过重依赖或过度抽象，短期看上去很完整，长期却会显著抬高演化成本。评审时要追问：模块接口是否清晰，内部细节是否封装得当，新依赖是否真正必要，设计有没有给未来变化留出合理空间。和惊艳式复杂方案相比，克制住炫技冲动通常更有价值。

清单解决的是“看什么”，流程解决的是“怎么做”。一个较稳妥的办法，是把 AI 时代的 PR 评审固定成五步闭环：先做需求对齐，确认目标、边界和验收条件；再做结构先行，优先查看模块划分和依赖方向；然后要求证据驱动，补齐最小测试和可复现样例；接着进行问题分级，把意见区分为必须修改、建议优化和可后续跟进；最后在必须项关闭后再复审合并，并记录关键决策。这套流程并不复杂，但能显著降低“语法正确而架构偏移”的风险。

在团队层面，还可以配置三条轻量机制来提高评审质量。第一，建立“拒绝清单”，例如跨层级混写、重复逻辑扩散、隐藏副作用、关键路径无测试等，只要命中就退回。第二，限制单次 PR 体量。AI 产出速度很快，PR 一旦过大，评审质量一定会下降。第三，沉淀“高频问题词典”，把常见评审意见模板化，减少重复沟通，让新人也能更快对齐标准。

提问阶段通过文档化 Prompt 约束生成空间，生成阶段通过复杂性预算控制输入速率，合并阶段再通过评审守住主干质量。三者互相补位，少了任何一个，系统风险都会被放大。在 AI 编程时代，代码产量已经不再稀缺，审查能力才是核心生产力。把每次 AI 输出都当作待合并 PR 来处理，用结构化评审守住正确性、可读性、可维护性和可演进性，项目才可能持续演化，不会滑进焦油坑。

第八章　协作公约

AI 提升个人产能，已经变成团队日常现实。一个工程师在短时间里完成过去数天工作量的情况并不罕见。原型速度更快，试错频率更高，方案覆盖面也明显扩展。问题也在同一时刻出现：团队只要缺少统一协作协议，高产出就会持续向系统输入未经对齐的实现。局部看，每一次提交都像是在推进；整体看，术语开始漂移，边界开始松动，依赖关系开始失控。代码总量增长很快，系统可解释性下降得更快。

《人月神话》对这一风险早有准确描述。进度灾难不只来自人手不足，也会来自沟通失序和组织失配。到了 AI 时代，这个结论更有现实感。团队面对的是一个持续产出实现草案的高频协作体。它能迅速给出局部解，但不会天然维护跨模块一致性。每个人只要各自和模型对话，各自形成隐性规则，团队很快就会进入一种“左手不知道右手在做什么”的状态，返工和冲突最终会在后期集中爆发。

个人提效要真正变成团队收益，就必须重新建立一套协作公约。这里说的“公约”，不是繁琐流程的堆叠，而是一组足够清楚、足够可执行的最低共识，让 AI 能力真正进入既有工程治理。

8.1　文本化与协议驱动

团队首先要把 Prompt 从个人技巧转变成公共资产。凡是影响核心业务逻辑、架构决策和跨模块行为的 Prompt，都应该以纯文本形式入库，并接受 Review。这样做的价值很直接：可追溯。任何实现结果都能追溯到输入假设，任何规则变化也都能通过版本历史解释来源。知识不再沉淀在私人聊天记录里，团队的学习成本和接手成本就会同步下降。

协议化还要求输入具备形式化结构。给 AI 的上下文，尽量包含明确接口契约、数据结构定义、错误语义和约束条件。自然语言叙述用于说明业务意图，结构化定义用于锁定可验证边界。两者同时存在时，生成结果通常更稳定。团队只给目标、不给边界，模型就会在默认假设中自行扩展实现，后续审查压力也会显著增加。

8.2　模块化隔离与边界控制

团队应该坚持“先定接口，再做实现”。架构师或模块负责人负责定义 API、依赖方向和数据流，AI 在既定接口内完成局部逻辑。这样做的好处很清楚：变动被控制在小范围里，每次生成都对应一个可验证单元。团队如果把整仓代码直接交给模型自由改写，影响面很快就会超出评审能力，质量治理也会随之失效。

边界控制还体现在最小上下文原则。每次调用 AI 时，只提供当前任务真正必需的信息：接口文件、相关数据模型、约束清单和目标测试。上下文越精准，输出通常越稳定；上下文越泛化，越容易引入跨层耦合和重复实现。减少无关上下文输入，不只是提问技巧，它本身就是工程能力。

8.3　精准调试与最小复现

AI 生成代码出现问题时，最有效的做法不是笼统地说“这里不对”，而是提供最小复现样例，并明确异常现象。笼统反馈只会得到笼统修补，问题很容易在下一轮以另一种形式出现。团队可以固定一套调试模板：输入条件、期望行为、实际行为、最小代码片段、日志和断言。模板化描述既帮助 AI 快速定位缺口，也帮助人类成员判断修复是否真正命中根因。

调试协议还必须和测试协议绑定。凡是涉及关键业务规则、状态转换和边界处理的 AI 产出，都要附带单元测试，并进入 CI 作为合并前置条件。测试在这里承担双重功能：验证当前正确性，限制后续回归风险。高频生成一旦脱离测试，只会把问题延后到集成阶段，再以更高成本一次性爆发。

8.4　评审权威与架构守门

团队应该把 AI 代码看成初级工程师提交的候选实现，合并前必须经过人工 Review。评审重点不在语法细节，而在概念完整性：术语是否一致，边界是否清晰，依赖是否合理，异常语义是否统一。只要破坏团队主线，就应该退回重做。评审标准越稳定，系统风格越稳定。

这套公约还要求团队明确角色分工：提交者负责解释生成路径和设计取舍，Reviewer 负责判断是否符合架构共识，模块负责人负责维护长期演进方向。三者职责清楚时，AI 的高速度和团队的高质量才有机会同时成立；职责一旦含混，所有人都只是在“补代码”，没有人在真正“守系统”。

为了让协作公约真正落地，可以固定两份模板。第一份是 AI 任务分配与上下文模板，至少包含背景目标、环境约束、接口契约、输入样例、禁止事项和验收标准。第二份是 AI 代码 PR 审查清单，至少覆盖接口透明度、单一职责、重复消除、边界处理和测试覆盖。模板化和清单化的意义，并不在于增加流程，而在于让质量门槛在高频提交中仍然保持稳定。

从方法论角度看，这套协作公约完成的是一次治理重心转移。AI 已经显著降低了构建成本，团队的关键任务转向了控制复杂度输入速率和维护概念完整性。只有把文本协议、模块边界、调试模板和评审机制连成闭环，个人层面的高产出才会真正转化成组织层面的长期收益。

第九章　收放之道

写到这里，有必要把视角从“如何更快生成代码”收回到“为什么要写这段代码、它准备活多久、由谁负责”。Vibe Coding 的价值已经很清楚：它大幅降低了起步成本，放大了试错速度，也让个人创造力更容易落地。问题同样清楚：任何方法只要失去边界，都会从效率工具变成风险来源。AI 时代真正成熟的开发者，是能够在自由探索和工程约束之间稳定切换的人。

这种切换并不矛盾。创造活动需要低成本草稿，工程活动需要高标准成品。Vibe Coding 解决的是草稿阶段的阻力，传统工程纪律解决的是成品阶段的可靠性。团队只有把草稿能力和成品要求放进同一条流程里，才能得到真正长期的收益。

从决策角度看，可以把开发现场粗略分成两类区域：探索区和生产区。探索区强调速度、反馈和学习；生产区强调一致性、可维护性和责任链路。很多项目最后失败，原因不在技术做不到，而在没有及时切换区域，让探索区的产物直接流入生产区。团队只要想避免这一点，就得明确：什么场景可以尽情 Vibe，什么场景必须切回工程纪律。

9.1　什么时候可以尽情 Vibe

第一类场景，是从零到一的原型验证。此时的目标，是快速确认方向是否成立，不是一次性得到可长期维护的系统。面向客户演示、产品可行性验证、交互路径试验，都属于这一类。这里可以容许代码粗糙，容许命名临时，容许局部重复。前提很明确：你知道它只是“验证材料”，后面还要重整。

第二类场景，是一次性脚本和小工具。比如批量数据清洗、临时格式转换、专项抓取、离线统计。这类代码运行周期短，边界窄，依赖关系简单。过度工程化只会增加成本。直接让 AI 生成并快速运行，通常是更优解。关键在于把这类产物明确标注为工具层，避免它们悄悄演化成核心依赖。

第三类场景，是学习与技术探边。接入陌生 API、试用新框架、理解第三方库边界时，可以让 AI 帮你快速生成实验代码和学习性测试。目标是缩短认知路径，尽快建立正确心智模型。探索代码要和生产代码保持隔离，避免实验性结论未经验证就进入主干。

9.2　什么时候必须切回工程纪律

第一类高风险场景，是代码进入团队协作。只要代码会被他人阅读、修改和扩展，就必须回到统一规范：命名一致、边界清晰、测试完整、审查严格。AI 生成代码合入主干前，必须经过 Code Review。评审关注点要落在可维护性和架构一致性上，不能停留在“能跑就行”。

第二类高风险场景，是触及核心业务链路。支付、结算、权限、核心算法、基础数据结构、公共协议这类代码，一旦出错，影响范围大，修复代价高。这里必须由人主导设计：先定接口和规则，再让 AI 实现局部细节。只要发现单一职责被破坏、重复规则出现、跨层依赖混入，就应该立刻重构，不能妥协合并。

第三类高风险场景，是复合问题调试和边界处理。系统出现幽灵 Bug、并发时序问题、异常链路错配时，不能依赖“反复让 AI 猜一次”的方式排障。最有效的路径，仍然是人主导定位：建立最小复现，补齐日志证据，写边界测试，逐段收敛。AI 可以参与局部修复建议，但不能替代问题定义和验证闭环。

9.3　一套可执行的切换规则

为了避免“该收不收、该放不放”，团队可以采用几条简单而明确的切换阈值。满足下面任意一条，就从 Vibe 模式切换到工程模式：

1. 代码预计存活超过一周，并且会被持续修改；
2. 代码将进入主干分支，或者会被跨团队复用；
3. 代码影响核心业务指标、资金安全、权限边界或用户数据；
4. 代码涉及异常处理、并发时序或复杂状态转换。

切换完成后，流程也要同步切换：补接口文档、补测试、补评审、补重构，不再把“生成速度”当成首要目标。

9.4　收束结论

真正的 Vibe Coder，能力不在于永远保持高速生成，而在于知道何时放开、何时收紧。起步阶段靠 Vibe 打开空间，降低心理门槛和实验成本；进入交付阶段，靠工程纪律保障质量，控制复杂度和责任风险。前者让想法更快出现，后者让结果得以长期存在。

把 AI 当成高效草稿引擎，把工程纪律当成最终发布标准。创造的自由当然值得保留，成品的责任也必须承担。只有这两件事同时成立，Vibe Coding 才会真正变成一种长期有效的开发方法。

结语

生成式人工智能迅速改变了编程现场，也迫使我们重新回答一个并不轻松的问题：当模型已经能够高频生成实现细节时，开发者究竟还剩下什么？

这个问题的答案，首先取决于我们怎么理解“开发”。如果把开发理解成“写代码”，那么 AI 的确已经替代掉了一部分旧有劳动。但开发本身远不止写代码。开发是一种持续实践，里面包括问题建模、结构约束、概念治理、风险控制和责任承担。把开发放回这个完整框架里，真正重要的部分不但没有消失，反而比过去更突出了。模型可以高频生成，团队就必须持续审阅；模型可以快速补全，开发者就必须维护概念完整性；模型可以降低起步门槛，人类就仍然要为系统的长期存续负责。

所以，最后的问题始终落在这里：生成之后，团队还能不能继续判断，继续约束，继续负责。一个团队只要能把 Prompt 当作设计，把评审当作基本功，把复杂性预算当作底线，把协作公约当作制度，把兴趣真正转化成责任，再把探索区和生产区清楚分开，AI 就不会只停留在令人兴奋的演示层面。它会真正变成一个长期有效的工程放大器。